

Testing the Critter Against Two Safety Standards

Test plan, conformance rubric, and the reasoning behind both

Artefacts under test	Six modules across the three Critter units, scored on six separate cards
Normative sources	SEI CERT C Coding Standard, 2016 Edition (v2016-06-29-1140) · NASA/JPL Power of 10 Rules (Holzmann, 2006)
Rules exercised	All 10 Power of 10 rules · 13 of the 99 numbered CERT rules
Suite	4 C test binaries, 2 Python checkers, 1 rubric engine · no external dependencies
Toolchain	GCC 13.3 · fanaalyzer · AddressSanitizer · UndefinedBehaviorSanitizer · python3

1. Scope, and what was deliberately left out

This document records the test plan for the three Critter units, the rubric that scores the results, and the reasoning behind each choice. It is a decision record, not a findings report — the findings are in §9, and the suite regenerates them on demand.

Two sources are used and no others: the SEI CERT C Coding Standard, 2016 Edition, and the NASA/JPL Power of 10 rules. No MISRA guidance, no JPL institutional standard, no CWE mapping, no outside commentary. Where those two sources leave a gap, the gap is recorded as a gap rather than filled from elsewhere; §8 collects three such cases, and they turn out to be among the more useful results here.

The exclusions are equally deliberate. `generate_temps.py` is a test-data generator that the memory unit’s own design record places outside the review boundary, so it is not scored. The test suite itself is not scored either: it is scaffolding, and scoring the instrument alongside the specimen would be circular. Its harness header says so explicitly, so a later reviewer cannot mistake the boundary.

2. Why thirteen CERT rules and not ninety-nine

The 2016 edition contains 99 numbered rules across fifteen chapters. Testing all of them against six small modules would produce a document that is mostly the phrase “no applicable construct”, and a reviewer cannot audit what they cannot read. The rules exercised here were selected against three criteria, applied in order:

1. **The construct is present.** The module must actually contain the thing the rule constrains. The concurrency chapter is untouched because none of these modules creates a thread.
2. **A violation is demonstrable, not arguable.** Preference went to rules where a test can produce evidence — a clobbered guard slot, a NaN command, two identical streams — over rules whose verdict would rest on a reviewer’s taste.
3. **The rule ID exists as a numbered section in the supplied edition.** This sounds like bookkeeping. It eliminated a rule the units’ own documentation relies on; see §8.1.

All ten Power of 10 rules are exercised on every applicable card. There are only ten, so completeness costs almost nothing, and the contrast between the two standards is more legible when one of them is covered exhaustively.

Chapter	Rules exercised	Why these, and why nothing else from the chapter
Declarations (DCL)	DCL31-C	Two units rely on identifiers that strict C11 does not declare. Caught by the compiler, so the check costs nothing and the evidence is a diagnostic.
Expressions (EXP)	EXP34-C	Unchecked allocation results appear in two modules and are correctly checked in a third, which makes the rule useful for discriminating between them rather than only for condemning.
Integers (INT)	INT30-C, INT31-C, INT33-C	The narrowing and wraparound cases. INT33-C is exercised specifically to record that it does <i>not</i> cover the divisions that matter here.
Floating point (FLP)	FLP30-C, FLP32-C	The compute unit takes a square root of a caller-supplied weight. FLP30-C is included as a control: it passes everywhere.
Arrays (ARR)	ARR30-C	The single most serious memory-safety finding in the product code.
Memory (MEM)	MEM35-C	Included because the compute unit gets it right; an all-failing rubric conveys no information.
Signals (SIG)	SIG31-C	The I/O unit installs a handler that shares state with its main loop — exactly the rule’s subject.
Error handling (ERR)	ERR33-C	The highest-volume finding across every card, and the one whose automation limits are most instructive.
Miscellaneous (MSC)	MSC30-C, MSC32-C	The simulated sensor is built on <code>rand()</code> with a fixed seed, which is both rules at once and a good test of whether the rubric can carry an intentional deviation.

3. Six cards, not three units

The assignment decomposes the Critter into three units. The suite scores six modules, because three of the shipped files declare themselves to be something other than product code and folding them together would corrupt every number downstream.

- `sensor_sim.c` sits behind the `sensor.h` hardware boundary, and the unit’s README calls it “the to-be-replaced part”. Its findings may be discharged by *deleting the file* when real hardware arrives. That is a categorically different disposition from

a finding in `io_unit.c`, which has to be fixed.

- `main.c` and `perf.c` are labelled “harness, not product” in their own header comments. The unchecked allocation in `main.c` is real, and it is also the kind of thing a benchmark driver gets away with.
- `mpc.c` and `thermal.c` are both product code but have very different profiles — one fails most of what it is measured against, the other passes most of it. A single compute-unit score would average that away and hide where the work is.

Cards are never merged. The rubric prints a cross-module summary but computes no unit-level or system-level aggregate, because there is no defensible way to average a card whose findings are architectural against one whose findings are typos.

4. Three test layers, assigned by rule semantics

Each rule is tested at the layer its own wording demands, not at whichever layer was convenient.

Layer	Implementation	Rules that belong here, and why
Runtime	Four C binaries, <code>test_*.c</code>	Rules whose violation has an observable signature: a wrong value, an out-of-bounds write, a lost data set, a NaN command. If a violation can be made to <i>happen</i> , a test that makes it happen is better evidence than a source scan that predicts it.
Static	<code>check_static.py</code> plus compiler and analyzer probes	Rules with no runtime signature at all. Power of 10 Rule 2 is the clearest case: it is violated when a tool <i>cannot prove</i> a bound, which is a property of the source text and cannot be observed by running anything. Rule 4 is function length; Rule 5 is assertion density; Rule 9 forbids function pointers whether or not they misbehave.
End-to-end	<code>check_dynamic.py</code>	Two jobs. First, independent confirmation: ARR30-C is caught in-process by guard slots and again by AddressSanitizer on the shipped binary, driven by a log file. Second, defects that only exist at the seam — the 128-byte line buffer is invisible from inside <code>parse_line</code> , which is handed a line and parses it correctly. That defect lives in the relationship between <code>count_lines</code> , <code>fgets</code> and the file, and only a real file shows it.

4.1 Reaching static functions without touching the source

Every function in `critter_mem.c` is static, so none of them can be called from another translation unit. Adding declarations, or removing static, would mean reviewing a file that is not the file under review. Instead the source is included into the test with its entry point renamed by the preprocessor:

```
#define main critter_mem_main
#include "critter_mem.c"
```

The bytes of the artefact are unchanged, and the rename happens in the test. The compute unit needs no such trick — its API has external linkage — and `thermal.c` is linked entirely alone, with a local stand-in for the workspace struct, so that a fault in its test cannot be attributed to `mpc.c`.

4.2 Testing a unit that has no testable entry point

The I/O unit’s only entry point is an unbounded acquisition loop terminated by a signal. It cannot be unit-tested without refactoring it, so it is tested the way it is used: started, run, interrupted, and audited through its CSV. The run length is chosen deliberately at six seconds, because at 100 Hz that crosses the 500-sample periodic-flush boundary exactly once, and a fencepost error in either the periodic flush or the tail flush would duplicate or drop samples precisely there. A shorter run would never reach that arithmetic.

4.3 Two tests that needed a process boundary

A test for missing parameter validation cannot run in-process, because the thing it detects is a crash. MPC-05 therefore calls the control step with a null observation in a forked child and has the parent read the child’s termination signal — the expected fault is *observed* rather than suffered. Similarly, MEM-01 detects an out-of-bounds subscript using guard slots that lie inside the same array object as the twelve the unit believes it was given, so the test can watch the overrun without itself committing undefined behaviour. A test for undefined behaviour that relies on undefined behaviour proves nothing.

5. The test plan

Every test carries an identifier, the rule it maps to, the mechanism that produces its verdict, and an explicit pass criterion. Tests expected to pass are marked; they are not padding. A rubric in which every check fails cannot distinguish a module that is careless from one that is merely unfinished, and three of the six cards here contain genuine passes that matter.

Card A — `unit1/io_unit.c`

Unit 1 — I/O · product code, in scope

Test	Source	Rule	Method	How it is tested	Pass criterion
I0-01	SEI CERT	SIG31-C	Compile probe	Redeclare the handler-shared object as <code>volatile sig_atomic_t</code> in the same translation unit and compile with <code>-fsyntax-only</code> .	Probe compiles. A conflicting-type diagnostic means the object is neither a lock-free atomic nor a <code>volatile sig_atomic_t</code> — the only two things the rule permits a handler to touch.
I0-02	SEI CERT	ERR33-C	Call-site audit + human	Classify every call to a function in ERR33-C's own table as discarded, assigned-untested, or checked.	Zero discarded results. Discarded is a definite violation; assigned-untested is reported with line numbers for a reviewer to close.
I0-03	SEI CERT	EXP34-C	gcc -fanalyzer	Static path analysis for null dereference on the <code>malloc/realloc</code> results.	Analyzer reports no null-dereference path.
I0-04	SEI CERT	DCL31-C	Compiler diagnostic	Compile at <code>-std=c11 -Wpedantic</code> and collect implicit-declaration and undeclared-identifier diagnostics.	Every identifier declared before use under the nominal standard.
I0-05	NASA P10	Rule 2	Source scan + human	Classify each loop bound as an integer literal or constant, an enclosing loop's induction variable, or a runtime value.	No loop bound is a runtime value. The rule counts a bound a tool cannot prove statically as violated.
I0-06	NASA P10	Rule 3	Source scan + human	Locate every allocator call and decide whether it sits inside a loop body.	No allocation, or all allocation confined to initialization. An allocator reached from a loop is a violation with no judgement required.
I0-07	NASA P10	Rule 5	Source scan	Count <code>assert()</code> calls and divide by the number of function definitions.	Average of at least two assertions per function.
I0-08	NASA P10	Rule 6	Source scan	Find file-scope object definitions at brace depth zero that lack <code>static</code> .	No file-scope object has external linkage.
I0-09	NASA P10	Rule 9	Source scan	Detect calls handing a function pointer to a library routine, function-pointer declarators, and declarations with two levels of indirection.	None of the three present.
I0-10	NASA P10	Rule 10	Compiler + analyzer	Build at the most pedantic setting available, then run <code>gcc -fanalyzer</code> .	Zero compiler warnings and zero analyzer warnings.
I0-11	NASA P10	Rule 4	Source scan	Brace-match every function and count its line span.	No function longer than 60 lines.
I0-12	Functional	Output integrity	Integration run	Run the unit for six seconds at 100 Hz, interrupt with <code>SIGINT</code> , audit the CSV. Six seconds crosses the 500-sample periodic-flush boundary once, which is the only way to reach the tail-flush arithmetic.	Correct header, seq contiguous from zero, no duplicates, no malformed rows, every value inside <code>int16_t</code> .
I0-13	Functional	Reproducibility	Integration run	Two seeded runs, compare the <code>raw_temp_c</code> column.	Identical prefixes. Every baseline-versus-hardened comparison in the review depends on this holding.

Card B — `unit1/sensor_sim.c`

Unit 1 — I/O · behind the `sensor.h` hardware boundary, scored apart

Test	Source	Rule	Method	How it is tested	Pass criterion
SIM-01	SEI CERT	MSC30-C	Runtime	Capture two 16-sample streams at the same seed and compare.	Streams differ. Identical streams confirm the generator is a seeded <code>rand()</code> , which is the rule's named subject.
SIM-02	SEI CERT	MSC32-C	Runtime	With <code>CRITTER_SEED</code> unset, capture two streams from separate <code>sensor_init()</code> calls.	Streams differ, which is the rule's own stated criterion for a properly seeded generator.
SIM-03	SEI CERT	INT31-C	Runtime	Request seed 2 ³² and seed 0 and compare the streams. This is the narrowing-conversion case: the value parses correctly, the destination type cannot hold it, nothing reports the loss.	Streams differ. Identical streams prove the <code>unsigned long → unsigned int</code> cast discarded the high 32 bits.
SIM-04	SEI CERT	ERR33-C	Runtime	Set <code>CRITTER_SEED</code> to an unparseable string; compare against seed 0.	The unit reports an error. Silently becoming seed 0 confirms <code>strtoul</code> 's documented error return is not checked.
SIM-05	SEI CERT	INT31-C	Runtime	200,000 reads; record the observed range against <code>int16_t</code> limits. Tests the <i>other</i> narrowing	Observed range inside <code>int16_t</code> . Recorded as not-applicable for this site so the INT31-

				site, the glitch return.	C failure is attributed to the seed cast alone.
SIM-06	NASA P10	Rule 2	Source scan	Loop inventory.	No unprovable bound. Recorded not-applicable: the module has no loops.
SIM-07	NASA P10	Rule 5	Source scan	Assertion density.	At least two per function.
SIM-08	NASA P10	Rule 6	Source scan	File-scope linkage.	No external-linkage object. The two file-scope statics are the smallest scope that can still hold state across a three-function interface.
SIM-09	NASA P10	Rule 10	Compiler + analyzer	Pedantic build.	Zero warnings from both.
SIM-10	NASA P10	Rule 4	Source scan	Function lengths.	All within 60 lines.
SIM-11	NASA P10	Rule 3	Source scan	Allocator inventory.	No dynamic allocation.

Card C — unit2/critter_mem.c

Unit 2 — memory · product code, in scope; generate_temps.py excluded per the unit's own design record

Test	Source	Rule	Method	How it is tested	Pass criterion
MEM-01	SEI CERT	ARR30-C	Runtime, guard slots	Hand summarize_by_month the interior of a 14-element array and feed it records with month 13 and month 0, then inspect the two slots either side. The test observes the overrun <i>without itself committing undefined behaviour</i> , because both guard slots are inside the same array object.	Guard slots unchanged. Either slot modified means the month field reached the subscript unchecked.
MEM-02	NASA P10	Rule 7	Runtime	Parse 13/45/2024 99:99:99,20.0.	Line rejected. Acceptance confirms seven successful conversions is the only criterion applied, which is what makes MEM-01 reachable from a log file.
MEM-03	SEI CERT	INT30-C	Runtime, 32-bit model	Model size_t as 32 bits and evaluate the unit's own allocation expression at 2 ²⁷ readings. This is the 'value does not fit the type' test: the arithmetic is right, the type is too narrow.	A guard fires before the multiplication wraps. On the Pi 5 the wrap is unreachable; the finding is a property of the type, not the board, and follows the source to any 32-bit userland.
MEM-04	SEI CERT	EXP34-C	Runtime	Force the compute_median allocation to fail.	Returns without dereferencing. Expected to pass — a suite that only ever reports violations cannot discriminate.
MEM-05	NASA P10	Rule 7	Runtime	Inspect what compute_median returns on allocation failure and what the caller can do with it.	Failure is distinguishable from success. Returning 0.0 °C — a plausible Celsius median — leaves the caller nothing to check.
MEM-06	NASA P10	Rule 7	Runtime	Forty identical readings, so the standard deviation is exactly zero, then call remove_outliers.	Readings survive, or the zero divisor is rejected. Every z-score becomes 0.0/0.0, and NaN fails the retention comparison.
MEM-07	SEI CERT	FLP30-C	Source scan	Loop-counter types.	No floating-point loop counter. Expected to pass.
MEM-08	SEI CERT	INT33-C	Human	Classify each division by a possibly-zero count.	Recorded not-applicable with the reason: all four are floating-point, and INT33-C constrains integer division only. See §8.
MEM-09	SEI CERT	ERR33-C	Call-site audit + human	Same audit as IO-02.	Zero discarded results.
MEM-10	NASA P10	Rule 2	Source scan + human	Same classification as IO-05.	No bound is a runtime value.
MEM-11	NASA P10	Rule 3	Source scan + human	Allocator inventory and loop-containment test.	All allocation confined to initialization. Sites outside a loop are reported for a reviewer to classify, because no tool can decide what counts as initialization.
MEM-12	NASA P10	Rule 4	Source scan	Function lengths.	All within 60 lines.
MEM-13	NASA P10	Rule 5	Source scan	Assertion density.	At least two per function.
MEM-14	NASA P10	Rule 9	Source scan	Function pointers and indirection depth.	None present.
MEM-15	NASA P10	Rule 10	Compiler + analyzer	Pedantic build.	Zero warnings from both. Expected to pass — and the pass is the point: every other finding on this card sits underneath a clean build.
MEM-16	SEI CERT	ARR30-C	End-to-end, sanitizers	Build the shipped source with AddressSanitizer and UndefinedBehaviorSanitizer and drive it with a log file containing one out-of-range month.	No sanitizer diagnostic. Independent confirmation of MEM-01 by a different mechanism: if the two disagree, the test is wrong, not the unit.
MEM-17	NASA P10	Rule 7	End-to-end	Run the shipped binary on a stuck-sensor log	The cleaned log is non-empty, or the unit

				and read its own report.	reports a fault.
MEM-18	—	Unmapped	End-to-end	One 153-character log line, arranged so the first record is padded to exactly 127 bytes and the second record begins precisely at the point fgets must stop.	Readings out equals physical lines in. Both halves parsing as valid records fabricates a reading that was never in the input.

Card D — unit3/mpc.c

Unit 3 — compute · product code, in scope

Test	Source	Rule	Method	How it is tested	Pass criterion
MPC-01	SEI CERT	FLP32-C	Runtime	Call one control step with <code>q_track = -1</code> , which the shipped harness accepts straight from <code>atoi(getenv(...))</code> , and inspect the returned plan.	Argument rejected before <code>sqrt</code> . The compounding failure to look for: the Cholesky guard is <code>d <= 1e-14</code> , and that comparison is false for NaN, so the guard does not fire.
MPC-02	—	Unmapped	Runtime	Compare the returned status byte against the returned payload.	Status is consistent with the payload it describes.
MPC-03	SEI CERT	MEM35-C	Runtime	After <code>crit_ws_alloc</code> , check that the last byte of every sub-array carved out of the arena lies inside it.	All ten extents within <code>[base, base+arena_bytes)</code> . Expected to pass.
MPC-04	NASA P10	Rule 2	Runtime	Solve an easy problem and a hard one; compare ADMM iteration counts.	Counts identical and equal to the compile-time constant. Expected to pass — the fixed iteration count is a deliberate design property.
MPC-05	NASA P10	Rule 7	Runtime, forked	Call the control step with a null observation pointer in a child process; the parent reads the child's termination signal. A test for missing parameter validation cannot run in-process, because the thing it detects is a crash.	Child exits normally. Termination by signal means none of the six pointer parameters is validated.
MPC-06	SEI CERT	DCL31-C	Compiler diagnostic	Compile at <code>-std=c11 -Wpedantic</code> .	No implicit declaration.
MPC-07	NASA P10	Rule 1	Source scan	Search for <code>goto</code> , <code>setjmp</code> , <code>longjmp</code> , and direct recursion.	None present.
MPC-08	NASA P10	Rule 2	Source scan + human	Loop-bound classification.	No bound is a runtime value. Bounds that are an enclosing loop's induction variable are reported as REVIEW , not failure — see §7.
MPC-09	NASA P10	Rule 4	Source scan	Function lengths.	All within 60 lines.
MPC-10	NASA P10	Rule 5	Source scan	Assertion density.	At least two per function.
MPC-11	NASA P10	Rule 10	Compiler + analyzer	Pedantic build.	Zero warnings from both.
MPC-12	NASA P10	Rule 3	Source scan + human	Allocator inventory.	Allocation confined to initialization. The single arena allocation is reported for reviewer classification rather than asserted to be a violation.

Card E — unit3/thermal.c

Unit 3 — compute · product code, in scope, linked alone so a fault here cannot be another module's

Test	Source	Rule	Method	How it is tested	Pass criterion
THM-01	NASA P10	Rule 2	Runtime	Run estimator initialization, which drives the Riccati recursion to its literal 4000-iteration bound, and inspect the resulting gain.	Terminates; all three gains finite; the measured-state gain lies in <code>(0, 1]</code> . Expected to pass.
THM-02	NASA P10	Rule 7	Runtime	One good measurement, then one NaN flagged valid, then ten more good measurements.	State stays finite, or the NaN is rejected. The <code>valid</code> flag says the reading is fresh; nothing says it is a number.
THM-03	SEI CERT	FLP32-C	Source scan	Inventory of <code><math.h></code> calls.	Recorded not-applicable: the file includes the header but calls nothing from it, so the rule has no construct to bite on.
THM-04	—	Unmapped	Runtime	Discretise two plants differing only in coil conductance; run both estimators under full cooling demand and compare trajectories.	Trajectories differ. Identical trajectories show the estimator's hardcoded conductances cannot track the plant it is estimating.
THM-05	NASA P10	Rule 1	Source scan	Control-flow inventory.	None present.
THM-06	NASA P10	Rule 2	Source scan	Loop-bound classification.	No unprovable bound.
THM-07	NASA P10	Rule 4	Source scan	Function lengths.	All within 60 lines.
THM-08	NASA P10	Rule 5	Source scan	Assertion density.	At least two per function.
THM-09	NASA P10	Rule 10	Compiler +	Pedantic build.	Zero warnings from both.

analyzer

THM-10	NASA P10	Rule 3	Source scan	Allocator inventory.	No dynamic allocation.
--------	----------	---------------	-------------	----------------------	------------------------

Card F — `unit3/perf.c` + `main.c`

Unit 3 — `compute` · `harness`, excluded from the unit's score — both files declare themselves so in their own headers

Test	Source	Rule	Method	How it is tested	Pass criterion
HRN-01	SEI CERT	EXP34-C	gcc -fanalyzer	Null-dereference path analysis.	No path reported.
HRN-02	SEI CERT	ERR33-C	Call-site audit + human	Audit across both files.	Zero discarded results.
HRN-03	NASA P10	Rule 8	Source scan	Count conditional-compilation directives outside include guards; detect token pasting and variadic macros.	Preprocessor use limited to includes and simple macros, with conditional compilation kept to a minimum.
HRN-04	NASA P10	Rule 9	Source scan	Function pointers and indirection depth.	None present.
HRN-05	NASA P10	Rule 1	Source scan	Control-flow inventory.	None present.

6. The rubric

The rubric scores two things and keeps them apart, because conflating them is how a review comes to claim a module is safe when what it actually means is that the checks were easy to run.

6.1 Conformance — pass or fail, per rule, per module

This is the score. A rule passes for a module only if every test mapped to that rule passed. One failure fails the rule.

Verdict	Meaning and effect on the score
PASS	The module satisfies the rule and a test demonstrates it. Counts toward conformance.
FAIL	The module violates the rule and a test demonstrates it. Counts against conformance.
REVIEW	The rule is neither satisfied nor violated on the available evidence, and closing it needs a human. Counts against conformance, because an open question is not a pass. See §7.
NA	The rule has no applicable construct in this module. Reported, but excluded from the denominator — a rule with nothing to bite on is neither passed nor failed. Recorded explicitly so “checked and clean” can be told apart from “never examined”.

Conformance = $\text{PASS} \div (\text{PASS} + \text{FAIL} + \text{REVIEW})$, over rules, not over tests. Several tests may map to one rule — ARR30-C is tested twice for the memory unit, from inside and from outside — and the rule takes the worst verdict of its tests.

6.2 Verification difficulty — what the verdict cost to obtain

Every test is tagged with the method that produced its verdict, and each method carries a cost in reviewer effort. The tag is assigned explicitly, test by test, in a table inside the rubric engine, so the cost model can be audited line by line rather than inferred.

Method	Cost	What it means, and why it is priced this way
COMPILE	1	The compiler or gcc -fanalyzer said so. Free, repeatable, no judgement, no maintenance. Put it in CI and stop thinking about it.
SCAN	2	A source scan decided it. Cheap to run, but somebody had to write the scanner, and its verdict is only as good as its heuristic. Counting assertions is reliable; deciding what a loop bound means is not.
RUNTIME	3	A test had to be designed, an input constructed, and the expected behaviour predicted in advance. This is where the engineering time actually goes. Constructing the split-line input required knowing the buffer size, the parser’s acceptance rule, and where fgets would stop.
HUMAN	5	No tool can close it. A reviewer must read the code and decide. Rule 3’s “after initialization”, Rule 2’s bounds that are real but unprovable, and ERR33-C’s assigned-but-untested residue all land here. Priced at five rather than four to reflect that this cost recurs on every change, while a CI check is paid once.

Two figures follow from the tags. **Effort** is the summed cost across a card’s tests — a rough estimate of what the card costs to verify once. **Manual share** is the fraction of a card’s findings whose cheapest available detection method was *not* the compiler: that is, how much of the review no tool will do for you.

6.3 Difficulty bands

Band	Trigger	What it tells the team
A — clean	No findings.	Re-run in CI and move on.
B — tool-assisted	Manual share $\leq$ 34% and conformance $\geq$ 60%.	Most findings came from the compiler or the analyzer, and local edits discharge them. Budget CI time.
C — reviewer-led	Manual share $\leq$ 67%.	The majority of findings needed a written test or a human reading. Budget reviewer time, not just CI time.
C+ — reviewer-led, heavy	Manual share $>$ 67%.	Almost nothing here is machine-detectable. This module sets the schedule.
D — redesign	A confirmed FAIL on Power of 10 Rule 2, 3 or 9.	At least one finding constrains the architecture — heap allocation after initialization, function pointers, or a loop with no provable bound. Hardening means rewriting the module, not patching it. Band D overrides the others: a module can be cheap to check and still need rebuilding.

A **REVIEW** on a structural rule deliberately does *not* trigger band D. A review is an open question, and answering it may clear the rule outright — which is exactly what happens to the compute unit’s Rule 3 verdict, where the single arena allocation looks like a violation to a scanner and is plainly initialization to a human.

6.4 Worked example

Card C, the memory unit, scores 12 applicable rules: 3 pass, 8 fail, 1 review, and one further rule recorded not-applicable and excluded. Conformance is $3 \div 12 = 25\%$. Of its 9 findings, none is reachable by the compiler or the analyzer — the module builds clean at the most pedantic setting and the analyzer reports nothing — so the manual share is 100%. That alone would place it in band C+. But its Rule 2 and Rule 9 verdicts are confirmed failures on structural rules, so band D overrides: the buffers are sized from the input file at runtime and the median goes through a library sort reached by function pointer, and neither is a local edit. Effort totals 53 points across 18 tests, the highest of any card, which is consistent with it being the only module whose findings all had to be constructed by hand.

7. Where the tools stop, and saying so

Power of 10 Rule 2 states that a bound a checking tool cannot prove statically counts as violated. Read naively, that licenses a lazy scanner to fail everything and call it rigour. The scanner here classifies each loop into three buckets instead of two:

- **Provable** — the bound is an integer literal, a macro constant, or an enumerator.
- **Review** — the bound is the induction variable of an enclosing loop in the same function. The loop is bounded, and proving it needs interval reasoning this scanner does not do. A stronger analyzer would likely discharge it; a human certainly can.
- **Unprovable** — the bound is a runtime value: a function parameter, a counter, a library call. No tool can prove this from the source, and no amount of tool quality changes that.

Only the third bucket produces a failure. The distinction matters because reporting the second bucket as a violation would blame the code for the scanner’s weakness, and any difficulty metric built on that is measuring the tool, not the artefact. It changes a real verdict: the compute unit’s MPC kernel has 28 of 32 loops bounded by compile-time constants and four bounded by an enclosing block index, and it is recorded as needing one reviewer pass rather than as failing.

The ERR33-C audit is honest in the same direction but cannot be conservative in the safe one. Deciding whether a stored return value is genuinely checked needs dataflow analysis. Call sites are therefore classified as *discarded* (the call is its own statement — a definite violation needing no dataflow), *assigned-untested* (stored, but the variable appears in no nearby comparison — a likely violation a human must confirm), or *checked*. Every site is printed with its line number so a reviewer can close the residue by eye. The size of that residue is itself a measurement, and it is what puts ERR33-C in the HUMAN cost bucket on four of six cards.

8. Three places the standards do not reach

8.1 A rule the units’ documentation cites that is not in this edition

The memory unit’s design record maps its divide-by-zero exposure to FLP03-C, twice. In the supplied 2016 edition, FLP03-C is not a rule. It appears only as a cross-reference inside the prose of other rules; it has no numbered section and no conformance criteria. Checking that a cited identifier actually exists in the normative source sounded like bookkeeping when it was written into the selection criteria in §2, and it caught a citation that a reviewer working from the design record would have accepted.

The substantive consequence is worse than the citation error. INT33-C covers divide-by-zero for *integer* division. Every division by a possibly-zero count in the memory unit is floating-point. So the 2016 edition contains **no rule** covering float divide-by-zero, and the finding has to be carried under Power of 10 Rule 7 — parameter validation — instead. The suite records this as an explicit not-applicable verdict with the reasoning attached, rather than quietly dropping it.

8.2 ERR33-C’s table is narrower than it looks

ERR33-C applies to the functions in its own table, and that table is specific. `fopen`, `fprintf`, `fclose`, `fflush`, `fgets`, `sscanf`, `malloc`, `realloc`, `signal`, `getenv` and `strtoul` are all in it, which makes the findings against them airtight. Plain `printf`, `puts`, `qsort`, `rewind` and `free` are *not*, so calls to them are not flagged — over-flagging would inflate the finding count and make the scorecard worthless. `nanosleep` and `clock_gettime` are POSIX rather than C standard and so are absent too, which means the I/O unit’s unchecked timing calls cannot be cited under this rule at all, despite being exactly the kind of thing it exists to prevent.

8.3 Three demonstrated defects with no rule in either standard

The suite found three real defects, each reproducible on demand, that no rule in either source covers. They are scored separately and excluded from conformance, because scoring a module against a rule that does not exist would be dishonest. They are the most interesting result in the exercise, because they are what a purely rule-driven review would have shipped.

Test	Module	Defect
MEM-18	<code>critter_mem.c</code>	A log line longer than the 128-byte buffer is split by <code>fgets</code> , and if the split falls in the right place <i>both halves parse as valid readings</i> . The suite constructs such a line and finds a temperature record in the cleaned output that appears in no input reading. STR31-C is not violated, because <code>fgets</code> does not overflow anything. No rule covers silent record fabrication by buffer splitting.
MPC-02	<code>mpc.c</code>	The control step returns status 0, meaning “ok”, alongside a NaN control command. Neither standard requires a status code to be consistent with the payload it describes, so no rule list would prompt a reviewer to check.
THM-04	<code>thermal.c</code>	The estimator hardcodes three conductances that duplicate plant-model fields, and takes no plant argument, so the duplication cannot be kept in step even in principle. Two plants differing only in coil conductance produce bit-identical estimator trajectories. Neither standard has a rule against duplicating a constant.

9. Results as measured

Reproduced verbatim from the suite. Numbers in this section are generated at render time from `build/verdicts.txt`, not transcribed, so the document cannot drift from the code.

```
=====
CRITTER SAFETY-GUIDELINE SCORECARD
Sources: SEI CERT C Coding Standard, 2016 Edition; NASA/JPL Power of 10 Rules
=====

-----
MODULE  unit2/critter_mem.c      [Unit 2 - memory / product]
-----
```

STANDARD	RULE	VERDICT	METHOD	TESTS
SEI CERT C	ARR30-C	FAIL	RUNTIME	MEM-01, MEM-16
NASA P10	Rule 7	FAIL	RUNTIME	MEM-02, MEM-05, MEM-06, MEM-17
SEI CERT C	INT30-C	FAIL	RUNTIME	MEM-03
SEI CERT C	EXP34-C	PASS	RUNTIME	MEM-04
SEI CERT C	FLP30-C	PASS	SCAN	MEM-07
SEI CERT C	INT33-C	NA	HUMAN	MEM-08
SEI CERT C	ERR33-C	FAIL	HUMAN	MEM-09
NASA P10	Rule 2	FAIL	HUMAN	MEM-10
NASA P10	Rule 3	REVIEW	HUMAN	MEM-11
NASA P10	Rule 4	FAIL	SCAN	MEM-12
NASA P10	Rule 5	FAIL	SCAN	MEM-13
NASA P10	Rule 9	FAIL	SCAN	MEM-14
NASA P10	Rule 10	PASS	COMPILE	MEM-15
(neither)	UNMAPPED	FINDING	RUNTIME	MEM-18

rules scored 12 pass 3 fail 8 review 1 not-applicable 1

CONFORMANCE 25% of applicable rules pass

FINDINGS 9 total: 0 compiler-detectable, 9 needing a written test or a human (100%)

UNMAPPED 1 demonstrated defect(s) with no rule in either standard

EFFORT 53 method-cost points across 18 tests

BAND D - redesign

at least one finding constrains the architecture (heap after init, function pointers, or unbounded loops), so hardening is a rewrite of this module, not a patch

MODULE unit3/mpc.c [Unit 3 - compute / product]

STANDARD	RULE	VERDICT	METHOD	TESTS
SEI CERT C	FLP32-C	FAIL	RUNTIME	MPC-01
SEI CERT C	MEM35-C	PASS	RUNTIME	MPC-03
NASA P10	Rule 2	REVIEW	HUMAN/RUNTIME	MPC-04, MPC-08
NASA P10	Rule 7	FAIL	RUNTIME	MPC-05
SEI CERT C	DCL31-C	FAIL	COMPILE	MPC-06
NASA P10	Rule 1	FAIL	SCAN	MPC-07
NASA P10	Rule 3	REVIEW	HUMAN	MPC-12
NASA P10	Rule 4	FAIL	SCAN	MPC-09
NASA P10	Rule 5	FAIL	SCAN	MPC-10
NASA P10	Rule 10	FAIL	COMPILE	MPC-11
(neither)	UNMAPPED	FINDING	RUNTIME	MPC-02

rules scored 10 pass 1 fail 7 review 2 not-applicable 0

CONFORMANCE 10% of applicable rules pass

FINDINGS 9 total: 2 compiler-detectable, 7 needing a written test or a human (78%)

UNMAPPED 1 demonstrated defect(s) with no rule in either standard

EFFORT 30 method-cost points across 12 tests

BAND C+ - reviewer-led, heavy

almost nothing here is machine-detectable; this module sets the schedule

MODULE unit3/thermal.c [Unit 3 - compute / product]

STANDARD	RULE	VERDICT	METHOD	TESTS
NASA P10	Rule 2	PASS	RUNTIME/SCAN	THM-01, THM-06
NASA P10	Rule 7	FAIL	RUNTIME	THM-02
SEI CERT C	FLP32-C	NA	SCAN	THM-03
NASA P10	Rule 1	PASS	SCAN	THM-05
NASA P10	Rule 3	PASS	SCAN	THM-10
NASA P10	Rule 4	PASS	SCAN	THM-07
NASA P10	Rule 5	FAIL	SCAN	THM-08
NASA P10	Rule 10	PASS	COMPILE	THM-09
(neither)	UNMAPPED	FINDING	RUNTIME	THM-04

rules scored 7 pass 5 fail 2 review 0 not-applicable 1

CONFORMANCE 71% of applicable rules pass

FINDINGS 2 total: 0 compiler-detectable, 2 needing a written test or a human (100%)

UNMAPPED 1 demonstrated defect(s) with no rule in either standard

EFFORT 19 method-cost points across 10 tests

BAND C+ - reviewer-led, heavy

almost nothing here is machine-detectable; this module sets the schedule

MODULE unit1/sensor_sim.c [Unit 1 - I/O / behind hardware boundary]

STANDARD	RULE	VERDICT	METHOD	TESTS
SEI CERT C	MSC30-C	FAIL	RUNTIME	SIM-01
SEI CERT C	MSC32-C	FAIL	RUNTIME	SIM-02
SEI CERT C	INT31-C	FAIL	RUNTIME	SIM-03, SIM-05
SEI CERT C	ERR33-C	FAIL	RUNTIME	SIM-04
NASA P10	Rule 2	NA	SCAN	SIM-06
NASA P10	Rule 4	PASS	SCAN	SIM-10
NASA P10	Rule 3	PASS	SCAN	SIM-11
NASA P10	Rule 5	FAIL	SCAN	SIM-07
NASA P10	Rule 6	PASS	SCAN	SIM-08
NASA P10	Rule 10	PASS	COMPILE	SIM-09

rules scored 9 pass 4 fail 5 review 0 not-applicable 1

CONFORMANCE 44% of applicable rules pass

FINDINGS 5 total: 0 compiler-detectable, 5 needing a written test or a human (100%)

EFFORT 26 method-cost points across 11 tests

BAND C+ - reviewer-led, heavy

almost nothing here is machine-detectable; this module sets the schedule

MODULE unit1/io_unit.c [Unit 1 - I/O / product]

STANDARD	RULE	VERDICT	METHOD	TESTS
SEI CERT C	SIG31-C	FAIL	COMPILE	IO-01
SEI CERT C	ERR33-C	FAIL	HUMAN	IO-02
SEI CERT C	EXP34-C	FAIL	COMPILE	IO-03
SEI CERT C	DCL31-C	FAIL	COMPILE	IO-04
NASA P10	Rule 2	FAIL	HUMAN	IO-05
NASA P10	Rule 3	FAIL	HUMAN	IO-06
NASA P10	Rule 4	FAIL	SCAN	IO-11
NASA P10	Rule 5	FAIL	SCAN	IO-07
NASA P10	Rule 6	FAIL	SCAN	IO-08
NASA P10	Rule 9	FAIL	SCAN	IO-09
NASA P10	Rule 10	FAIL	COMPILE	IO-10
Functional	OUTPUT-IN	PASS	RUNTIME	IO-12
Functional	REPRODUCI	PASS	RUNTIME	IO-13

rules scored 11 pass 0 fail 11 review 0 not-applicable 0

CONFORMANCE 0% of applicable rules pass

FINDINGS 11 total: 4 compiler-detectable, 7 needing a written test or a human (64%)

EFFORT 27 method-cost points across 13 tests

BAND D - redesign

at least one finding constrains the architecture (heap after init, function pointers, or unbounded loops), so hardening is a rewrite of this module, not a patch

MODULE unit3/perf.c+main.c [Unit 3 - compute / harness, excluded]

STANDARD	RULE	VERDICT	METHOD	TESTS
SEI CERT C	EXP34-C	FAIL	COMPILE	HRN-01
SEI CERT C	ERR33-C	FAIL	HUMAN	HRN-02
NASA P10	Rule 8	FAIL	SCAN	HRN-03
NASA P10	Rule 9	FAIL	SCAN	HRN-04
NASA P10	Rule 1	PASS	SCAN	HRN-05

rules scored 5 pass 1 fail 4 review 0 not-applicable 0

CONFORMANCE 20% of applicable rules pass

FINDINGS 4 total: 1 compiler-detectable, 3 needing a written test or a human (75%)

EFFORT 12 method-cost points across 5 tests

BAND D - redesign

at least one finding constrains the architecture (heap after init, function pointers, or unbounded loops), so hardening is a rewrite of this module, not a patch

=====

CROSS-MODULE SUMMARY (cards are never merged; each is scored against its own role)

=====

MODULE	UNIT	RULES	PASS	CONF	FINDS	MANUAL	EFFORT	BAND
unit2/critter_mem.c	Unit 2 - memory	12	3	25%	9	100%	53	D - redesign
unit3/mpc.c	Unit 3 - compute	10	1	10%	9	78%	30	C+ - reviewer-led, heavy
unit3/thermal.c	Unit 3 - compute	7	5	71%	2	100%	19	C+ - reviewer-led, heavy
unit1/sensor_sim.c	Unit 1 - I/O	9	4	44%	5	100%	26	C+ - reviewer-led, heavy
unit1/io_unit.c	Unit 1 - I/O	11	0	0%	11	64%	27	D - redesign
unit3/perf.c+main.c	Unit 3 - compute	5	1	20%	4	75%	12	D - redesign

ACROSS ALL CARDS: 54 rule verdicts scored, 14 pass (26%), 37 fail, 3 review, 3 not applicable.

3 demonstrated defects have no rule in either standard.

Total verification effort: 167 method-cost points.

9.1 Every verdict, with its recorded evidence

Module	Test	Rule	Verdict	Evidence recorded by the suite
unit2/critter_mem.c	MEM-01	ARR30-C	FAIL	month=13 wrote summary[12]:yes month=0 wrote summary[-1]:yes (12-element array, no range check at critter_mem.c:319)
unit2/critter_mem.c	MEM-02	Rule 7	FAIL	parse_line accepted month=13 day=45 hour=99 min=99 sec=99; 7 sscanf conversions is the only acceptance criterion
unit2/critter_mem.c	MEM-03	INT30-C	FAIL	32-bit size_t: 134217728 readings x 32 B wraps to 0; a guard (n > SIZE_MAX/elem) would have caught it. malloc then succeeds on a 0-byte request and read_readings walks off the buffer. critter_mem.c:471,472 perform no such check
unit2/critter_mem.c	MEM-04	EXP34-C	PASS	compute_median returns on allocation failure without dereferencing the null result
unit2/critter_mem.c	MEM-05	Rule 7	FAIL	compute_median signals allocation failure by returning 0.0 degC, indistinguishable from a valid median; caller at critter_mem.c:526 has no way to detect it
unit2/critter_mem.c	MEM-06	Rule 7	FAIL	constant input: std_dev=0.0, z=NaN, kept 0 of 8 readings; downstream mean=NaN. remove_outliers does not validate std_dev (critter_mem.c:260)
unit2/critter_mem.c	MEM-07	FLP30-C	PASS	loop counters are long/int throughout; verified by inspection of all 9 loops and by check_static.py
unit2/critter_mem.c	MEM-08	INT33-C	NA	every division by a possibly-zero count in this unit (compute_mean:181, compute_std_dev:194, summary mean:382,412) is floating-point. INT33-C constrains integer division only. The 2016 edition contains no rule for float divide-by-zero: FLP03-C appears only as a cross-reference inside FLP32-C, not as...
unit3/mpc.c	MPC-01	FLP32-C	FAIL	q_track=-1 -> sqrt(-1) at mpc.c:186,313 -> u_now[0]=NaN, and every NaN comparison being false leaves the sentinels in place: t_peak=-1e+30 degC, margin_c=1e+30 degC. status=0 ("ok"). The Cholesky guard d<=1e-14 does not fire on NaN
unit3/mpc.c	MPC-02	UNMAPPED	FINDING	crit_mpc_step returns status=0 ("ok") alongside a NaN command. Neither standard has a rule requiring a status code to be consistent with the payload it describes; a reviewer working only from rule lists will not be prompted to look for this
unit3/mpc.c	MPC-03	MEM35-C	PASS	arena 451424 B holds Gh(96x192) + H(192x192) + 5 M-vectors + yfree + Ad + Bd with 512 B spare; all extents inside [base, base+arena_bytes)
unit3/mpc.c	MPC-04	Rule 2	PASS	T_air=22.5 and T_air=45.0 both ran exactly 25 ADMM iterations (CRIT_ADMM_K); iteration count is input-independent
unit3/mpc.c	MPC-05	Rule 7	FAIL	crit_mpc_step(obs=NULL) terminated by signal 11; the observation pointer is dereferenced at mpc.c:299 before any validity check. 6 of 6 pointer parameters unvalidated
unit3/thermal.c	THM-01	Rule 2	PASS	kalman_steady bound is the literal 4000 (thermal.c:150); terminated with L=[0.132198, 0.033943, 2.045294], all finite, L[0] in (0,1]
unit3/thermal.c	THM-02	Rule 7	FAIL	one NaN measurement with valid=1 poisoned the state; 10 subsequent good readings did not recover it (x=[NaN, NaN, NaN]). crit_est_update checks o->valid but not isfinite(o->t_air) (thermal.c:214)
unit3/thermal.c	THM-03	FLP32-C	NA	thermal.c calls no <math.h> function; all arithmetic is + - * / on double. The #include <math.h> at thermal.c:12 is unused
unit3/thermal.c	THM-04	UNMAPPED	FINDING	ua_coil=950 and ua_coil=0 both give T_air_hat=24.690472153 after 20 updates at full cooling demand: crit_est_update uses the literals 950.0/260.0/120.0 (thermal.c:207-209) and takes no crit_plant_t argument, so it cannot track the plant it is estimating. Neither standard has a rule against duplicati...
unit1/sensor_sim.c	SIM-01	MSC30-C	FAIL	rand() at sensor_sim.c:45,54,55; seed 777 reproduces the identical 16-sample stream (first=2195 last=2202). The whole sensor output is determined by one 32-bit seed
unit1/sensor_sim.c	SIM-02	MSC32-C	FAIL	no CRITTER_SEED: two runs produced identical streams (first=2201). sensor_sim.c:24 defaults to the literal seed 12345. DOCUMENTED DEVIATION: README requires repeatability for baseline-vs-hardened diffing
unit1/sensor_sim.c	SIM-03	INT31-C	FAIL	CRITTER_SEED=4294967296 (2^32) produced the same stream as CRITTER_SEED=0: (unsigned int)strtoul(...) at sensor_sim.c:28 discards the high 32 bits. sizeof(unsigned long)=8, sizeof(unsigned int)=4
unit1/sensor_sim.c	SIM-04	ERR33-C	FAIL	CRITTER_SEED="not-a-number" silently became seed 0 (identical stream, first=2201). strtoul is in ERR33-C's function table; sensor_sim.c:28 passes NULL for endptr and checks neither the return value nor errno
unit1/sensor_sim.c	SIM-05	INT31-C	NA	200000 reads spanned [1144, 3201] hundredths degC, inside int16_t [-32768, 32767]. The int32->int16 cast at sensor_sim.c:55 is bounded

by the [1500,3500] clamp; the INT31-C FAIL for this module rests on the seed cast (SIM-03), not this one

unit1/io_unit.c	I0-01	SIG31-C	FAIL	'running' is declared 'int' at line 29 and is read by handler handle_sigint and by the main loop. Redefining it 'volatile sig_atomic_t' is rejected by the compiler, so it is neither a lock-free atomic nor a volatile sig_atomic_t: the two types the rule permits
unit1/io_unit.c	I0-02	ERR33-C	FAIL	11 call(s) to ERR33-C-table functions: 7 DISCARDED [io_unit.c:60 signal, io_unit.c:68 fprintf, io_unit.c:94 fprintf, io_unit.c:97 fflush, io_unit.c:99 fflush, io_unit.c:107 fprintf, io_unit.c:111 fclose]; 4 ASSIGNED-UNTESTED [io_unit.c:63 fopen->out, io_unit.c:65 fopen->out, io_unit.c:72 malloc...
unit1/io_unit.c	I0-03	EXP34-C	FAIL	gcc -fanalyzer reports 1 null-dereference path(s): io_unit.c:87 possible-null-dereference
unit1/io_unit.c	I0-04	DCL31-C	FAIL	-std=c11: implicit declarations ['clock_gettime', 'nanosleep']; undeclared identifiers ['CLOCK_REALTIME']. The unit relies on identifiers the selected standard does not declare; it builds only because the default dialect is a GNU one
unit1/io_unit.c	I0-05	Rule 2	FAIL	3 of 3 loops have a bound no tool can prove from the source (runtime values): while line 80 'running'; for line 93 'i = count - FLUSH_EVERY; i < count; i++'; for line 106 'i = count - (count % FLUSH_EVERY); i < count; i = 0 provable, 0 bounded by an enclosing loop ...
unit1/io_unit.c	I0-06	Rule 3	FAIL	2 dynamic allocation site(s) [malloc:72, realloc:84], 1 of them inside a loop body (realloc in main). An allocation reached repeatedly from a loop is unambiguously after initialization, so no judgement is needed to call this a violation
unit1/io_unit.c	I0-11	Rule 4	FAIL	1 of 3 functions exceed 60 lines: main=67
unit1/io_unit.c	I0-07	Rule 5	FAIL	0 assert() call(s) across 3 functions = 0.00 per function; Rule 5 requires an average of at least 2. Shortfall 6 assertions
unit1/io_unit.c	I0-08	Rule 6	FAIL	1 file-scope object(s) with external linkage: 'running' (int) line 29. Adding 'static' would restrict each to this translation unit at no cost
unit1/io_unit.c	I0-09	Rule 9	FAIL	1 call(s) passing a function pointer to a library routine: io_unit.c:60 signal(); 1 declaration(s) with two levels of indirection: io_unit.c:51 'char **argv'; Rule 9 allows at most one level of dereference
unit1/io_unit.c	I0-10	Rule 10	FAIL	does not COMPILE under -std=c11 -Wall -Wextra -Wpedantic: 1 error(s), 3 warning(s). First: /home/claude/critter/unit1_io/io_unit.c:47:19: error: 'CLOCK_REALTIME' undeclared (first use in this function)
unit1/sensor_sim.c	SIM-06	Rule 2	NA	module contains no loops
unit1/sensor_sim.c	SIM-10	Rule 4	PASS	all 3 functions within 60 lines (longest 21)
unit1/sensor_sim.c	SIM-11	Rule 3	PASS	no dynamic memory allocation anywhere in the module
unit1/sensor_sim.c	SIM-07	Rule 5	FAIL	0 assert() call(s) across 3 functions = 0.00 per function; Rule 5 requires an average of at least 2. Shortfall 6 assertions
unit1/sensor_sim.c	SIM-08	Rule 6	PASS	no file-scope object has external linkage; 2 file-scope object(s) declared static, which is the smallest scope that can still hold state across calls
unit1/sensor_sim.c	SIM-09	Rule 10	PASS	-std=c11 -Wall -Wextra -Wpedantic: zero warnings; gcc -fanalyzer: zero warnings
unit2/critter_mem.c	MEM-09	ERR33-C	FAIL	26 call(s) to ERR33-C-table functions: 17 DISCARDED [critter_mem.c:222 fprintf, critter_mem.c:352 fprintf, critter_mem.c:357 fprintf, critter_mem.c:363 fclose, critter_mem.c:374 fprintf, critter_mem.c:378 fprintf, critter_mem.c:385 fprintf, critter_mem.c:393 fprintf, critter_mem.c:398 fclose, critte...
unit2/critter_mem.c	MEM-10	Rule 2	FAIL	9 of 12 loops have a bound no tool can prove from the source (runtime values): while line 127 'fgets(line, (int)sizeof(line), file) != NULL'; while line 156 'stored < capacity && fgets(line, (int)sizeof(l); for line 178 'i = 0; i < count; i++'; for line 19...
unit2/critter_mem.c	MEM-11	Rule 3	REVIEW	3 dynamic allocation site(s) [malloc:217, malloc:471, malloc:472], none inside a loop. Rule 3 forbids allocation AFTER initialization and permits it during, so the verdict turns on whether each site is reached only at startup -- which no tool can decide. A reviewer must classify each site; that judg...
unit2/critter_mem.c	MEM-12	Rule 4	FAIL	1 of 14 functions exceed 60 lines: main=117
unit2/critter_mem.c	MEM-13	Rule 5	FAIL	0 assert() call(s) across 14 functions = 0.00 per function; Rule 5 requires an average of at least 2. Shortfall 28 assertions
unit2/critter_mem.c	MEM-14	Rule 9	FAIL	1 call(s) passing a function pointer to a library routine: critter_mem.c:230 qsort(); 1 declaration(s) with two levels of indirection: critter_mem.c:432 'char **argv'; Rule 9 allows at most one level of dereference
unit2/critter_mem.c	MEM-15	Rule 10	PASS	-std=c11 -Wall -Wextra -Wpedantic: zero warnings; gcc -fanalyzer: zero

				warnings
unit3/mpc.c	MPC-06	DCL31-C	FAIL	-std=c11: implicit declarations ['posix_memalign']; undeclared identifiers []. The unit relies on identifiers the selected standard does not declare; it builds only because the default dialect is a GNU one
unit3/mpc.c	MPC-07	Rule 1	FAIL	restricted control flow: goto at line 265
unit3/mpc.c	MPC-08	Rule 2	REVIEW	28 of 32 loops are bounded by compile-time constants; the remaining 4 are bounded by an enclosing loop's induction variable (for line 148 'int j0 = 0; j0 <= i0; j0 += 4'; for line 212 'int k = 0; k < j; k++;'; for line 219 'int k = 0; k < j; k++;'; for line ...
unit3/mpc.c	MPC-12	Rule 3	REVIEW	1 dynamic allocation site(s) [posix_memalign:43], none inside a loop. Rule 3 forbids allocation AFTER initialization and permits it during, so the verdict turns on whether each site is reached only at startup -- which no tool can decide. A reviewer must classify each site; that judgement is the cost...
unit3/mpc.c	MPC-09	Rule 4	FAIL	1 of 9 functions exceed 60 lines: crit_mpc_step=82
unit3/mpc.c	MPC-10	Rule 5	FAIL	0 assert() call(s) across 9 functions = 0.00 per function; Rule 5 requires an average of at least 2. Shortfall 18 assertions
unit3/mpc.c	MPC-11	Rule 10	FAIL	-std=c11: 1 compiler warning(s) ['implicit-function-declaration']
unit3/thermal.c	THM-05	Rule 1	PASS	no goto, no setjmp/longjmp, no direct recursion
unit3/thermal.c	THM-06	Rule 2	PASS	all 13 loops bounded by compile-time constants
unit3/thermal.c	THM-10	Rule 3	PASS	no dynamic memory allocation anywhere in the module
unit3/thermal.c	THM-07	Rule 4	PASS	all 7 functions within 60 lines (longest 34)
unit3/thermal.c	THM-08	Rule 5	FAIL	0 assert() call(s) across 7 functions = 0.00 per function; Rule 5 requires an average of at least 2. Shortfall 14 assertions
unit3/thermal.c	THM-09	Rule 10	PASS	-std=c11 -Wall -Wextra -Wpedantic: zero warnings; gcc -fanalyzer: zero warnings
unit3/perf.c+main.c [harness]	HRN-01	EXP34-C	FAIL	gcc -fanalyzer reports 1 null-dereference path(s): main.c:107 possible-null-dereference
unit3/perf.c+main.c [harness]	HRN-02	ERR33-C	FAIL	11 call(s) to ERR33-C-table functions: 2 DISCARDED [main.c:77 fprintf, main.c:114 fprintf]; 1 ASSIGNED-UNTESTED [main.c:100 malloc->us]; 8 checked. The discarded set is a definite violation; the assigned-unttested set needs human confirmation
unit3/perf.c+main.c [harness]	HRN-03	Rule 8	FAIL	5 conditional-compilation directive(s) outside include guards (perf.c has 5); Rule 8 requires these be kept to a minimum
unit3/perf.c+main.c [harness]	HRN-04	Rule 9	FAIL	1 call(s) passing a function pointer to a library routine: main.c:116 qsort(); 1 declaration(s) with two levels of indirection: main.c:47 'char **argv'; Rule 9 allows at most one level of dereference
unit3/perf.c+main.c [harness]	HRN-05	Rule 1	PASS	no goto, no setjmp/longjmp, no direct recursion
unit2/critter_mem.c	MEM-16	ARR30-C	FAIL	AddressSanitizer on the shipped source, driven by a log file: month_13.txt -> stack-buffer-overflow; month_00.txt -> stack-buffer-underflow. Independently confirms MEM-01, which used guard slots in-process
unit2/critter_mem.c	MEM-17	Rule 7	FAIL	40 identical readings (stuck sensor): read 40, reported 40 as outliers, wrote 0 to the cleaned log. The unit exits 0 and reports success while deleting the entire data set
unit2/critter_mem.c	MEM-18	UNMAPPED	FINDING	9 physical lines became 10 readings (0 malformed reported). One 153-character line exceeded the 128-byte buffer; fgets split it at 127 bytes and BOTH halves parsed as valid, so the record 01/02/2023 03:04:05.99.900 -- which is in no input reading -- appears in critter_cleaned.txt. No rule in either ...
unit1/io_unit.c	I0-12	OUTPUT-INTEGRITY	PASS	6.0 s at 100 Hz produced 592 samples, seq contiguous 0..591, no duplicates, no malformed rows, all values inside int16_t. Periodic-flush boundary crossed (>500 samples)
unit1/io_unit.c	I0-13	REPRODUCIBILITY	PASS	two runs at CRITTER_SEED=20260916 agree on the first 198 raw_temp_cC values, so baseline-vs-hardened diffing on the seq/raw_temp_cC columns is sound as the README claims

10. Reading the results

Three observations that the rubric was built to surface, and did.

The compiler is nearly useless here, and that is the headline. Across all six cards, the great majority of findings were not reachable by the compiler or by gcc -fanalyzer. The memory unit is the extreme case: it builds clean at -Wall -Wextra -Wpedantic -std=c11 with zero warnings, the analyzer reports nothing, and every one of its nine findings had to be constructed by hand. Its own design record claims the clean build, and the claim is true — which is the point. Every finding on that card sits underneath it.

The two standards fail in different directions, and the split is structural. The CERT findings are mostly local and mostly mechanical once you know where to look: a missing NULL check, a narrowing cast, an unchecked return. The Power of 10 findings are mostly architectural — heap allocation after initialization, a library sort reached by function pointer, loop counts driven by input size, zero assertions anywhere. No analyzer will tell a team that heap allocation after initialization is forbidden, because that is a policy about architecture rather than a detectable defect. That asymmetry is why band D is triggered only by Power of 10 rules.

Two units do not compile at the standard they nominally target. The I/O unit does not build at all under `-std=c11 -Wpedantic` — `CLOCK_REALTIME` is a hard error — and the MPC kernel emits an implicit declaration for `posix_memalign`. Both build fine under the default GNU dialect, which is why nobody noticed. Power of 10 Rule 10 asks for a pedantic build from the first day of development, and this is what that rule is for. The suite is built `-std=gnu11` as a consequence, and that concession is recorded as a finding against the units rather than absorbed silently into the build system.

11. Limits of this suite

- **The rubric weights are a judgement.** The 1/2/3/5 cost model is defensible but not derived from measurement. Anyone re-running this should feel free to re-price it; the tags are separated from the costs precisely so that re-pricing does not require re-tagging.
- **Conformance treats all rules as equal.** An out-of-bounds array write and a missing assertion each cost one rule. Severity weighting was left out deliberately, because a severity scale is a second set of contestable judgements layered on the first, and the per-rule table gives a reader everything needed to apply their own.
- **The static scanner is a heuristic, not a parser.** It strips comments and literals and matches braces; it does not build an AST. It is accurate on these six files, each of which was checked by hand against its output, and two false results were found and fixed during development — a false negative on `DCL31-C` caused by GCC quoting identifiers with typographic quotation marks, and a false positive on Rule 6 caused by struct members being mistaken for file-scope objects. On a larger codebase it would need to become a real front end.
- **Absence of a sanitizer report is not absence of a defect.** A 12-element automatic array can be overrun without landing outside any sanitizer redzone. This is why `ARR30-C` is tested twice by different mechanisms, and why the end-to-end check reports **REVIEW** rather than **PASS** if the sanitizers stay quiet while the guard-slot test fires.
- **Thirteen of ninety-nine CERT rules.** The selection is justified in §2 and each choice is auditable, but it is a selection. A conformance figure of 25% means 25% of the rules *tested*, and should never be quoted without that qualifier.

12. Reproducing this

No network access and no packages beyond a compiler and python3.

```
cd tests
make run      # build, run all three layers, print the scorecard
make verdicts # raw CT| verdict lines, unscored
make runtime  # the four C test binaries only
make static   # structural and compile probes only
make dynamic  # end-to-end and integration only (~10 s)
python3 make_report.py # regenerate this document from the latest run
```

Source locations are overridable: `make U1=./io-unit U2=./mem-unit U3=./compute-unit`. Fault inputs are written to `faults/` and left in place on purpose, so a reviewer can read the exact bytes that produced each verdict. Every verdict is one line of the form `CT|module|test|standard|rule|verdict|evidence`, which is the only interface between the three layers and the rubric — adding a check means emitting one more line, not modifying the scorer.